

Obstacles for a Llama

يريد لاما السفر عبر هضبة الأنديز. لديه خريطة للهضبة على شكل شبكة من $N \times M$ خلية مربعة. صفوف الخريطة مرقمة من 0 إلى $N - 1$ من الأعلى إلى الأسفل، وأعمدتها مرقمة من 0 إلى $M - 1$ من اليسار إلى اليمين. الخلية في الصف i والعمود j ($0 \leq i < N, 0 \leq j < M$) يُرمز لها (i, j) .

درس اللاما مناخ الهضبة واكتشف أن جميع الخلايا في كل صف من الخريطة لها نفس درجة الحرارة، وجميع الخلايا في كل عمود من الخريطة لها نفس الرطوبة. أعطتك اللاما مصفوفتين بأعداد صحيحة T و H بطول N و M على التوالي. هنا، يشير $T[i]$ ($0 \leq i < N$) إلى درجة حرارة الخلايا في الصف i ، ويشير $H[j]$ ($0 \leq j < M$) إلى رطوبة الخلايا في العمود j .

لقد درس اللاما أيضًا نباتات الهضبة ولاحظ أن الخلية (i, j) تكون خالية من النباتات إذا وفقط إذا كانت درجة حرارتها أكبر من رطوبتها، أي $T[i] > H[j]$.

لا يمكن للاما السفر عبر الهضبة إلا باتباع مسارات صالحة. المسار الصالح هو سلسلة من الخلايا المختلفة التي تستوفي الشروط التالية:

- يشترك كل زوج من الخلايا المتتالية في المسار في جانب مشترك.
- جميع الخلايا في المسار خالية من النباتات.

مهمتك هي الإجابة على Q سؤال. لكل سؤال، لديك أربعة أعداد صحيحة: L, R, S, D . يجب عليك تحديد ما إذا كان هناك مسار صحيح بحيث:

- يبدأ المسار من الخلية $(0, S)$ وينتهي عند الخلية $(0, D)$. من المضمون أن يكون كل من $(0, S)$ و $(0, D)$ خاليًا من النباتات.
- تقع جميع الخلايا في المسار ضمن الأعمدة L إلى R ، بما في ذلك.

Implementation Details

The first procedure you should implement is

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : مصفوفة من الطول N تحدد درجة الحرارة في كل صف.
- H : مصفوفة من الطول M تحدد الرطوبة في كل عمود.
- يتم استدعاء هذا الإجراء مرة واحدة فقط لكل حالة اختبار، قبل أي استدعاءات إلى `can_reach`.

The second procedure you should implement is

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : الأعداد الصحيحة التي تصف السؤال.
- يتم استدعاء هذا الإجراء Q مرة لكل حالة اختبار.

يجب أن يعود هذا الإجراء true إذا فقط في حالة وجود مسار صحيح من الخلية $(0, S)$ إلى الخلية $(0, D)$, بحيث تقع جميع الخلايا في المسار ضمن الأعمدة من L إلى R محتواه.

Constraints

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq i < N$ for each i such that $0 \leq T[i] \leq 10^9$
- $0 \leq j < M$ for each j such that $0 \leq H[j] \leq 10^9$
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- كلتا الخليتين $(0, S)$ و $(0, D)$ خالية من النباتات.

Subtasks

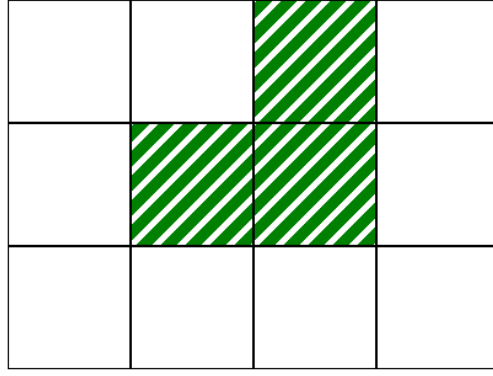
Subtask	Score	Additional Constraints
1	10	$L = 0, R = M - 1$ for each question. $N = 1$.
2	14	$L = 0, R = M - 1$ for each question. $T[i - 1] \leq T[i]$ for each i such that $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ for each question. $N = 3$ and $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ for each question. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ for each question.
6	17	No additional constraints.

Example

:Consider the following call

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

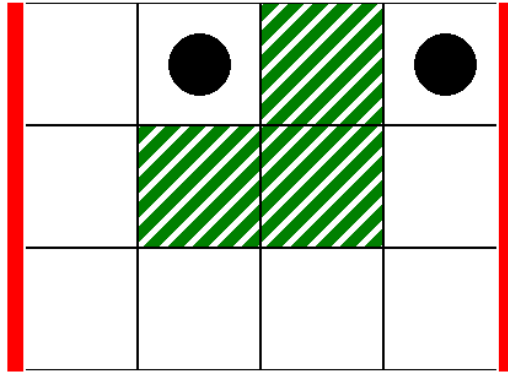
وهذا يتوافق مع الخريطة في الصورة التالية، حيث الخلايا البيضاء خالية من النباتات:



:As the first question, consider the following call

```
can_reach(0, 3, 1, 3)
```

يتوافق هذا مع الوضع في الصورة التالية، حيث تشير الخطوط العمودية السمكية إلى مجال الأعمدة من $L = 0$ إلى $R = 3$ ، وتشير الأقراص السوداء إلى خلايا البداية والنهاية:



في هذه الحالة، يمكن أن تصل اللاما من الخلية $(0, 1)$ إلى الخلية $(0, 3)$ من خلال المسار الصحيح التالي:

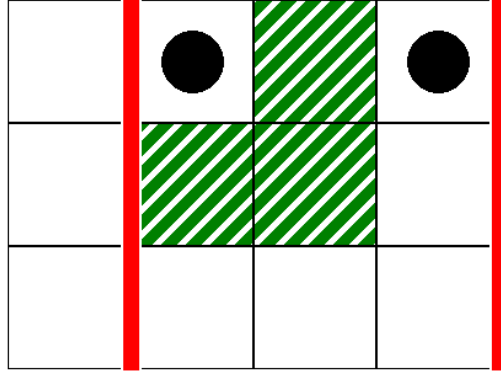
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

لذلك، يجب أن تعود هذه المكالمات `true`.

بالنسبة للسؤال الثاني، انظر إلى المكالمات التالية:

```
can_reach(1, 3, 1, 3)
```

هذا يتوافق مع السيناريو في الصورة التالية:



في هذه الحالة، لا يوجد مسار صالح من الخلية $(0, 1)$ إلى الخلية $(0, 3)$ ، بحيث تقع جميع الخلايا في المسار داخل الأعمدة 1 إلى 3، بما في ذلك. لذلك، يجب أن تعود هذه المكالمة `false`.

Sample Grader

:Input format

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

هنا، $L[k], R[k], S[k], D[k]$ ($0 \leq k < Q$) تحدد المعطيات لكل استدعاء إلى `can_reach`.

:Output Format

```
A[0]
A[1]
...
A[Q-1]
```

هنا، $A[k]$ ($0 \leq k < Q$) هو 1 إذا كانت المكالمات `can_reach(L[k], R[k], S[k], D[k])` عادت `true`، و 0 خلاف ذلك.